DLDCA Lab 5: Stack Overflow

Week 6

September 4, 2026

Introduction

This lab moves from integers to floating point, and introduces real functions. The tasks build up from scalar floating-point arithmetic, through writing and composing your own functions under the `SystemV` calling convention, to recursion, along with an additional graded task.

Necessary tools

We shall be using `nasm` to assemble x86-64 assembly into object files, and `ld/gcc/g++` to link those object files into an executable. Before starting the lab, ensure both are installed:

```
$ nasm -v
NASM version [version]
$ ld -v # or gcc/g++ --version
GNU ld (GNU Binutils for Ubuntu) [version]
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- task1/
    |- dotprod.s (TODO)
|- task2/
    |- callee_caller.s (TODO)
    |- dump_regs.s
    |- to_a.s
|- task3/
    |- power.s (TODO)
|- labexam/
    |- poly_eval.s (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz` in lowercase. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Background: Scalar Floating Point & Functions

x86-64 provides sixteen 128-bit `xmm` registers. For this lab, we only care about the low 64 bits of each register (enough to hold a `double`) and only the *scalar double* (`sd`) instruction forms, such as `movsd`, `addsd`, and `mulsd`. The *packed* forms (`ps`/`pd`) operate on several values at once and will be covered in the next lab.

As a reminder, the following facts hold true about the `SystemV` convention:

1. Integer/pointer arguments and floating-point arguments are assigned from two *separate* sequences of registers – the first six integer/pointer arguments go in `rdi, rsi, rdx, rcx, r8, r9`, while the first eight floating-point arguments go in `xmm0` through `xmm7`, independently of each other. A function taking a `float` and an `int` would put the `float` in `xmm0` and the `int` in `rdi`.
2. Return values follow the same split: an integer/pointer result comes back in `rax`, a floating-point result in `xmm0`.
3. Every `xmm` register is caller-saved. There is no floating-point equivalent of a callee-saved general-purpose register like `rbx/r1[2-5]`. If you need a value in an `xmm` register to survive a `call`, you are responsible for saving it yourself (the stack is the usual place).
4. Syscalls are like special functions. They go to the `OS` and tell it to perform some action. To utilize them, we put the 'syscall number' in `rax` and put arguments in `rdi, rsi, rdx, r10, r8, r9`

Tasks

Task 1: Floating-Point Dot Product

The same dot product from your very first assembly lab, except `vec1` and `vec2` now hold 64-bit doubles instead of integers, and the arithmetic needs to happen in the `xmm` registers instead of the general-purpose ones. With the values provided, their dot product

$$\texttt{vec1}[0] \cdot \texttt{vec2}[0] + \texttt{vec1}[1] \cdot \texttt{vec2}[1] + \cdots + \texttt{vec1}[n-1] \cdot \texttt{vec2}[n-1]$$

comes out to 11.3. Since the exit code has to be an integer, truncate this to its integer part toward zero using `cvttsd2si` (round towards zero).

Fill in the body of `_start` in `dotprod.s`.

To run your code:

```
$ make task1
...
Return Code: 11
```

Task 2: Caller and Callee Semantics

Let's now write functions which are actually callable using `call` and return using `ret`. We will follow the `SystemV` convention throughout this lab. You are given `dump_regs()`, a function that prints every general-purpose register's current value and modifies nothing; calling it should look exactly like a no-op to the rest of your program. You'll use it to directly observe what a function call does and doesn't change.

(You can also use gdb instead of putting calls to `dump_regs` in your code.)

`func_callee`

Implement `func_callee(int a{rdi}, int b{rsi}) -> rax`, which returns `a + b`. Once you've computed the sum, deliberately overwrite every *other* caller-saved register (`rcx`, `rdx`, `rsi`, `rdi`, `r8`, `r9`, `r10`, `r11`) with any junk value you like. This models a "worst case" callee – one that gives you no guarantees beyond what the ABI actually promises.

`func_caller`

Implement `func_caller() -> rax`. It needs to call `func_callee(10, 20)`, but it also needs a multiplier of `3` to survive that call, so that it can return `func_callee`'s result times `3` afterward.

Call `dump_regs()` once right before calling `func_callee`, and once right after it returns, so you have a before-and-after snapshot of every register to compare.

Putting it together

In `_start`, call `func_caller()` and exit with its result.

Before you move on: look at the two `dump_regs` outputs side by side. Which registers changed, and which didn't? For each one, why? Is it because the ABI guarantees it? Is it because `func_callee` happened to leave it alone anyway? Or is it something else? In particular, check what happened to whichever register you chose to keep your multiplier in.

Fill in `func_callee`, `func_caller`, and `_start` in `callee_caller.s`.

To run your code:

```
$ make task2
rax: 0    rbx: 3    rcx: 0    ...
rax: 30   rbx: 3    rcx: 57005 ...
Return Code: 90
```

Task 3: Recursive Power, Two Ways

You're given a double `base` and a non-negative integer `exponent` in `.data`. This task asks for two different implementations of the same function, so you can see the difference between ordinary recursion and a tail call directly.

3a: Naive recursion

Implement `pow_naive(double base{xmm0}, uint64_t exponent{rdi}) -> xmm0` using *exponentiation by squaring*, i.e. the recurrence

$$b^e = \begin{cases} 1 & e = 0 \\ (b \times b)^{e/2} & e \text{ even} \\ b \times (b \times b)^{(e-1)/2} & e \text{ odd} \end{cases}$$

using ordinary `call`/`ret` recursion.

A reminder about `SystemV`'s conventions:

1. `base` is a double, so it's passed in `xmm0`, the first *floating-point* argument register. `exponent` is an integer, so it's passed in `rdi`, the first *integer* argument register. Register locations for `double pow(double, uint64_t)` and `double pow(uint64_t, double)` will be identical.
2. `xmm0` does not survive a `call` unless you save it first. The odd case above needs the original `base` *after* the recursive call returns, by which point `xmm0` has been reused for the call's argument and return value.

3b: Tail-call rewrite

Implement `pow_tail(double base{xmm0}, uint64_t exponent{rdi}, double acc{xmm1}=1.0) -> xmm0`, computing the exact same result, but rewritten such that we use `jmp` to go back to the function. Thread an accumulator through `xmm1` (call it initially with `acc = 1.0`) so that the multiplication which used to happen *after* the recursive call in 3a now happens *before* it instead.

Implement this using `jmp` instead of `call`/`ret` for the recursive step. To see what changes, step through a run of your program and see the value of `rsp`. What do you observe?

Putting it together

In `_start`, call both `pow_naive(base, exponent)` and `pow_tail(base, exponent, 1.0)`, truncate each result toward zero, and exit with their sum. Since both should independently come out to `57`, a correct solution exits with `114`.

Fill in `pow_naive`, `pow_tail`, and `_start` in `power.s`.

To run your code:

```
$ make task3
...
Return Code: 114
```

Lab Exam: Polynomial Evaluation

You are given an array of doubles, `coeffs`, its length `coeffs_len`, and a double `x_val`. Treating `coeffs` as the coefficients of a polynomial (`coeffs[0]` is the constant term, `coeffs[1]` the coefficient of x^1, and so on), evaluate

$$\texttt{coeffs}[0] \cdot x^0 + \texttt{coeffs}[1] \cdot x^1 + \cdots + \texttt{coeffs}[n-1] \cdot x^{n-1}$$

at $x =$ `x_val`, truncate the result toward zero, and exit with that as the exit code.

You'll implement this recursively, using Horner's method – which just means factoring x out repeatedly:

$$c_0 + x(c_1 + x(c_2 + \dots))$$

so that each step only needs one multiply and one add. As a recursive function over "the coefficients from some starting point onward", this is:

```
poly_eval(coeffs, n, x):
    if n == 1:
        return coeffs[0]

    rest = poly_eval(coeffs + 8, n - 1, x)
    return coeffs[0] + x * rest
```

Implement this as `poly_eval(double* coeffs{rdi}, size_t N{rsi}, x{xmm0})`.

Fill in `poly_eval` and `_start` in `poly_eval.s`.

To run your code:

```
$ make labexam
...
Return Code: 49
```

Table 1: Some basic x86-64 Assembly Instructions

Instruction	Operands	Effect
`mov`	`reg, reg/*mem/imm`	`reg` ← `reg/*mem/imm`
`mov`	`*mem, reg`	`*mem` ← `reg`
`movzx / movsx`	`reg, reg/*mem`	`reg` ← `reg/*mem`, zero-/sign-extended
`lea`	`reg, *mem`	`reg` ← address of `*mem`
`add`	`reg, reg/*mem/imm`	`reg` ← `reg` + `reg/*mem/imm`
`add`	`*mem, reg/imm`	`*mem` ← `*mem` + `reg/imm`
`sub`	`reg, reg/*mem/imm`	`reg` ← `reg` − `reg/*mem/imm`
`sub`	`*mem, reg/imm`	`*mem` ← `*mem` − `reg/imm`
`inc`	`reg/*mem`	`reg/*mem` ← `reg/*mem` + 1
`dec`	`reg/*mem`	`reg/*mem` ← `reg/*mem` − 1
`neg`	`reg/*mem`	`reg/*mem` ← −`reg/*mem` (Two's complement)
`not`	`reg/*mem`	`reg/*mem` ← ¬`reg/*mem` (One's complement/bitwise)
`shl / shr`	`reg/*mem, imm/cl`	Logical shift left / right by `imm` or `cl` bits
`sar`	`reg/*mem, imm/cl`	Arithmetic shift right (preserves sign bit)
`imul`	`reg, reg/*mem/imm`	`reg` ← `reg` × `reg/*mem/imm`
`imul`	`*mem, reg/imm`	`*mem` ← `*mem` × `reg/imm`
`div`	`reg`	`rax` ← `(rdx:rax)`/`reg` `rdx` ← `(rdx:rax)`%`reg`
`and`	`reg, reg/*mem/imm`	`reg` ← `reg` ∧ `reg/*mem/imm`
`and`	`*mem, reg/imm`	`*mem` ← `*mem` ∧ `reg/imm`
`or`	`reg, reg/*mem/imm`	`reg` ← `reg` ∨ `reg/*mem/imm`
`or`	`*mem, reg/imm`	`*mem` ← `*mem` ∨ `reg/imm`
`xor`	`reg, reg/*mem/imm`	`reg` ← `reg` ⊕ `reg/*mem/imm`
`xor`	`*mem, reg/imm`	`*mem` ← `*mem` ⊕ `reg/imm`
`cmp`	`reg/*mem, reg/imm`	Sets flags based on subtraction: `reg/*mem` - `reg/imm`. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
`test`	`reg/*mem, reg/imm`	Sets flags based on bitwise AND: `reg/*mem` ∧ `reg/imm`. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
`jmp`	`label`	Unconditional jump to label
`je / jz`	`label`	Jump if Zero Flag (ZF) = 1
`jne / jnz`	`label`	Jump if Zero Flag (ZF) = 0
`jl`	`label`	Jump if Less (SF ≠ OF)
`jle`	`label`	Jump if Less or Equal (ZF = 1 or SF ≠ OF)
`jg`	`label`	Jump if Greater (ZF = 0 and SF = OF)
`jge`	`label`	Jump if Greater or Equal (SF = OF)
`cmov`*cc*	`reg, reg/*mem`	`reg` ← `reg/*mem` if *cc* holds, else unchanged.
`syscall`		Performs a syscall with syscall number stored in `rax` Arguments passed in other registers starting from `rdi`
`push`	`reg`	Pushes the value in `reg` onto the stack
`pop`	`reg`	Pops the value on top of the stack onto `reg`
`call`	`label`	Calls the function called `label`
`ret`		Pops the return address from stack and jumps to it

Table 2: Scalar Floating-Point (`xmm`) Instructions

Instruction	Operands	Effect
`movsd`	`xmm, xmm/*mem`	`xmm` ← `xmm/*mem` (low 64 bits)
`movsd`	`*mem, xmm`	`*mem` ← `xmm` (low 64 bits)
`movq`	`xmm, reg/*mem`	`xmm` ← `reg/*mem` (raw 64 bits, no conversion)
`movq`	`reg/*mem, xmm`	`reg/*mem` ← `xmm` (raw 64 bits, no conversion)
`xorps / xorpd`	`xmm, xmm`	`xmm` ← `xmm` $\oplus$ `xmm` (idiom to zero an `xmm` register)
`pxor`	`xmm, xmm`	`xmm` ← `xmm` $\oplus$ `xmm` (bitwise XOR on `xmm`)
`addsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` + `xmm/*mem`
`subsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` − `xmm/*mem`
`mulsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` × `xmm/*mem`
`divsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` ÷ `xmm/*mem`
`comisd`	`xmm, xmm/*mem`	Sets flags like `cmp`, comparing as doubles (use `ja/jae/jb/jbe`). Raises exception on NaN.
`ucomisd`	`xmm, xmm/*mem`	Same as `comisd`, but does not raise exception on NaN
`cvttsd2si`	`reg, xmm/*mem`	`reg` ← integer part, truncated toward zero
`cvtsi2sd`[1]	`xmm, reg/*mem`	`xmm` ← `reg/*mem` converted to a double

[1] Prefer loading floating point literals or the binary representation directly instead of using this. Use this only for runtime `int` to `double` conversions